

Proposed RAG (Retrieval-Augmented Generation) Architecture

Introduction

To improve the factual accuracy and reliability of Large Language Model (LLM) responses, the proposed system adopts a **Retrieval-Augmented Generation (RAG)** architecture. Unlike a standalone LLM that relies only on its pre-trained knowledge, RAG combines an LLM with an external knowledge base. Before generating a response, the system retrieves relevant information from stored documents and supplies it to the LLM as context. This approach significantly reduces hallucinations and improves the quality, relevance, and trustworthiness of AI-generated responses.

Proposed RAG Architecture

The proposed architecture consists of the following components:

Step 1: Document Collection

The system collects documents from trusted sources such as:

- PDF documents
- Technical reports
- Research papers
- Company documentation
- Knowledge base articles

These documents form the knowledge base of the system.

Step 2: Document Preprocessing

Before storing the documents, preprocessing is performed to improve retrieval quality.

The preprocessing steps include:

- Text extraction from PDF and Word files
 - Removal of unnecessary formatting
 - Cleaning special characters
 - Normalization of whitespace
 - Metadata extraction (document name, page number, source)
-

Step 3: Text Chunking

Since LLMs cannot process very large documents efficiently, documents are divided into smaller sections called **chunks**.

Each chunk contains meaningful information while preserving the context.

The chunks are then stored individually for efficient retrieval.

Step 4: Embedding Generation

Each text chunk is converted into a numerical vector called an **embedding**.

Embeddings capture the semantic meaning of the text rather than just individual words.

These vectors enable similarity search between the user's query and stored documents.

Step 5: Vector Database

The generated embeddings are stored inside a Vector Database.

Unlike traditional databases that search by keywords, vector databases perform **semantic similarity search**, allowing the system to retrieve documents with similar meaning.

Step 6: Query Processing

When a user asks a question:

1. The question is converted into an embedding.
 2. The vector database searches for the most similar document chunks.
 3. The top relevant chunks are retrieved.
-

Step 7: Context Injection

The retrieved document chunks are combined with the user's query.

This additional information is provided as context to the LLM.

Step 8: Response Generation

The LLM generates a response using:

- User Question
- Retrieved Context
- Internal Language Understanding

Because the answer is grounded in retrieved documents, the chances of hallucination are significantly reduced.

Step 9: Response Evaluation

The generated response is evaluated using:

- Semantic similarity
- Reference-grounded evaluation
- LLM-as-a-Judge
- Human evaluation (if required)

Only high-quality responses are presented to the user.

Embedding Model Used

Proposed Embedding Model

"Sentence Transformers (all-MiniLM-L6-v2)"

Why this embedding model?

The proposed project uses the **all-MiniLM-L6-v2** sentence embedding model because it provides an excellent balance between accuracy, speed, and computational efficiency.

The model converts text into dense vector representations that preserve semantic meaning, making it highly suitable for document retrieval in RAG systems.

Advantages

- Produces high-quality semantic embeddings.
- Fast embedding generation.
- Low computational requirements.
- Excellent performance for semantic search.
- Widely used in production RAG applications.
- Easy integration with Python-based AI frameworks.

This embedding model is particularly suitable for educational and prototype projects due to its lightweight architecture and strong retrieval performance.

Vector Database Used

Proposed Vector Database

"FAISS (Facebook AI Similarity Search)"

Why FAISS?

FAISS is an open-source vector database developed by Meta for efficient similarity search over high-dimensional vectors.

It enables rapid retrieval of the most relevant document chunks based on semantic similarity.

Advantages

- Extremely fast vector search.
- Efficient memory utilization.
- Supports millions of vector embeddings.
- Easy integration with LangChain and Python.
- Ideal for local development without cloud infrastructure.
- Widely adopted in Retrieval-Augmented Generation systems.

For Milestone 1, FAISS is selected because it is lightweight, free, and well-suited for prototype development.

Chunking Strategy Used

Proposed Strategy

"Recursive Character Text Chunking"

Description

Recursive Character Text Chunking divides large documents into smaller, meaningful text chunks while preserving sentence and paragraph boundaries whenever possible.

Instead of splitting text at fixed positions, the algorithm recursively checks for natural separators such as:

- Paragraphs
- Line breaks

- Sentences
- Spaces

This results in more coherent chunks and improves retrieval quality.

Proposed Configuration

- Chunk Size: **500 characters**
- Chunk Overlap: **100 characters**

The overlap ensures that important information spanning adjacent chunks is not lost, preserving context during retrieval.

Advantages

- Maintains semantic continuity.
- Prevents loss of important contextual information.
- Improves retrieval accuracy.
- Reduces fragmented responses.
- Compatible with LangChain and most RAG frameworks.

Overall RAG Workflow

1. User uploads documents.
2. Documents are preprocessed.
3. Documents are divided into chunks.
4. Each chunk is converted into vector embeddings using the **all-MiniLM-L6-v2** embedding model.
5. Embeddings are stored in the **FAISS Vector Database**.
6. The user's query is converted into an embedding.
7. FAISS retrieves the most semantically similar document chunks.
8. Retrieved chunks are passed as context to the Large Language Model.
9. The LLM generates a grounded response.
10. The generated response is evaluated using semantic similarity and reference-grounded evaluation before being presented to the user.

Proposed Technology Stack

Component	Selected Technology	Purpose
Large Language Model	Gemini API	Response Generation
Embedding Model	Sentence Transformers (all-MiniLM-L6-v2)	Semantic Embeddings
Vector Database	FAISS	Semantic Search

Component	Selected Technology	Purpose
Chunking Strategy	Recursive Character Text Splitter	Document Segmentation
Framework	LangChain	RAG Pipeline
Backend	Python + Flask	API Development
Frontend	React.js	User Interface

Conclusion

The proposed Retrieval-Augmented Generation (RAG) architecture combines semantic document retrieval with Large Language Models to generate accurate, context-aware, and trustworthy responses. By integrating the **all-MiniLM-L6-v2 embedding model**, **FAISS vector database**, and **Recursive Character Text Chunking** strategy, the system achieves efficient document retrieval while minimizing hallucinations. This architecture provides a scalable, modular, and industry-standard foundation for building reliable AI applications and serves as an appropriate design choice for the Milestone 1 prototype.